

Vibe Coding AI 全栈开发新范式

传统开发

- 语法难度
- 究极码农
- 代码驱动
- 实现细节优先

AI 赋能

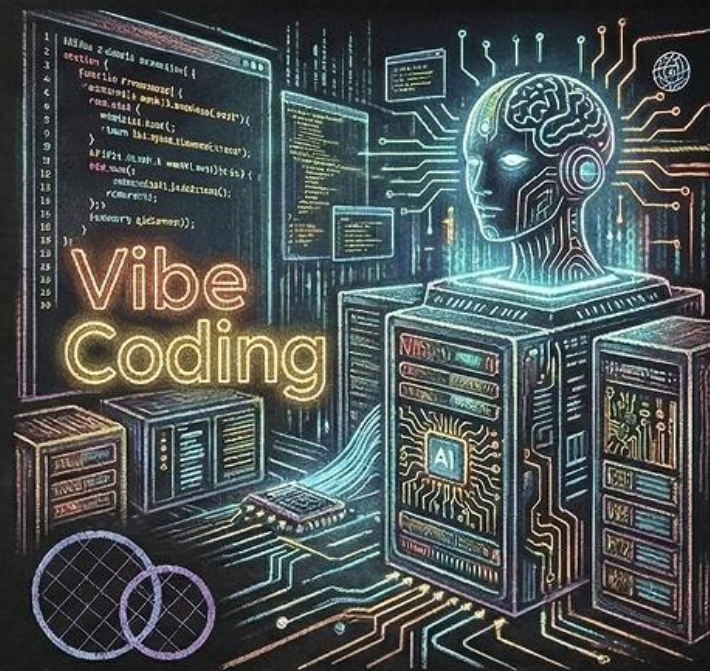


Vibe Coding

- 直达产品价值
- 产品主理人
- 意图驱动
- 用户体验优先

跨越语法障碍，直达产品价值；从“码农”进化为“产品主理人”

不写一行代码，交付工业级产品



初识 Vibe Coding

氛围编码 / 氛围编程

什么是 Vibe Coding?

一种新兴的软件开发范式或风格

- 不是传统意义上手写代码
- 通过自然语言对 AI 进行指导
- 让 AI 自动生成、修改和迭代代码
- 由 Andrej Karpathy 于 2025 年提出



实践场景举例

快速原型设计

内部工具/小 App 开发

创新性实验代码

非工程背景人员
参与开发沟通



Vibe Coding 并不是要完全替代传统软件工程，
而是简化开发流程、加速创新、提升协作效率。

简化流程

加速创新

提升协作

Andrej Karpathy · 2025



Vibe Coding 核心特征

01 自然语言驱动

用自然语言描述需求, AI 自动生成代码
“帮我写一个 TODO List 的 Web 应用”
→ AI 生成 HTML、CSS、JavaScript

02 AI 作为主要编码者

AI 不只是自动补全, 而是核心代码生成者
开发者角色转变: 定义意图 → 确认结果
→ 调整输出, 而非逐行编写代码

03 交互性与快速迭代

对话式、循环式的开发过程
提需求 → AI 输出 → 检查测试 → 反馈调整
快速循环, 敏捷实验想法

04 降低技术门槛

AI 负责繁琐技术细节
无传统编程经验也能构建应用
软件开发民主化, 更贴近产品想法

05 目标与结果导向

不关注代码“怎么写”, 关注能否实现功能
意图驱动、结果导向的思维转变
不纠结实现细节, 聚焦最终效果

核心理念

Vibe Coding 让开发者从“写代码”转变为“指导 AI”, 专注于创意和产品价值, 而非技术实现细节。这是编程方式的根本性变革。

Vibe Coding 到底是难还是简单？

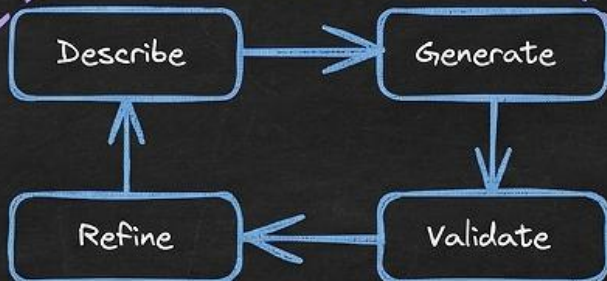
😊 看起来简单

不需要写代码语法
用自然语言描述需求
AI 自动生成代码

低门槛快速产出
简单页面、小工具、原型
几句话实现功能

✓ 少受语法阻碍
✓ 快速看到成果

🔄 迭代循环本质



不是线性，而是迭代

😞 实际很难

自然语言有歧义
Prompt 设计是一项技能

AI 输出不稳定
需要反复验证和调整

需要理解并修正代码
检查、调试甚至重写

复杂系统需要工程能力

💡 核心理解

👤 新手

“看起来很简单”——模糊技术细节
让人更快看到结果

👤 资深工程师

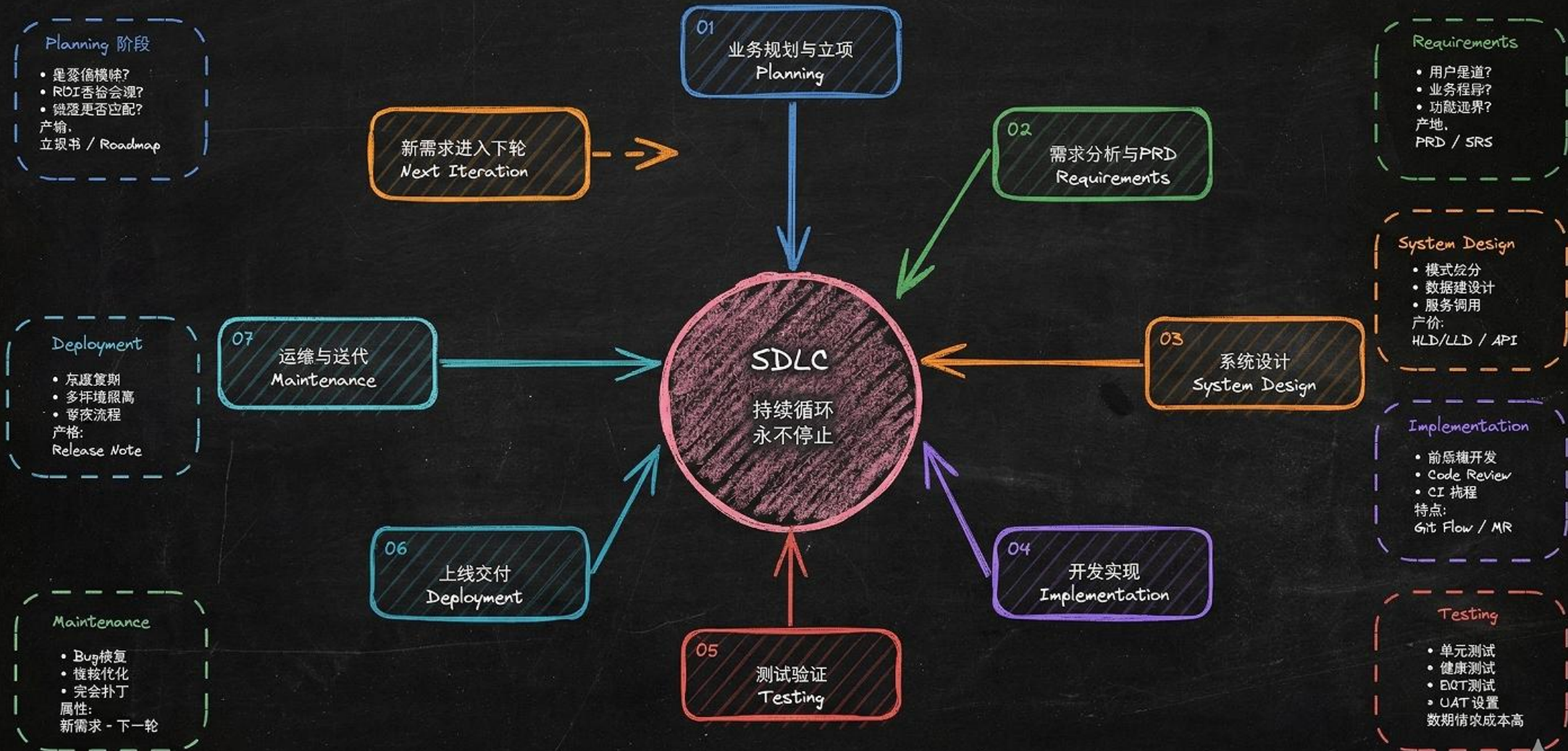
“并不简单”——生产级软件需处理
安全性、可维护性、架构设计

☆ 现实价值定位

快速原型、创意验证，提高生产率
而非完全取代传统软件工程

简单演示 → 复杂业务逻辑 → 可维护架构 → 高质量软件工程
需要掌握：Prompt 设计、调试方法、底层编程、架构知识

Enterprise SDLC 软件开发生命周期



传统企业研发 = 强规划 + 强文档 + 强流程 + 强审批 -> 稳定但慢, 适合高合规场景

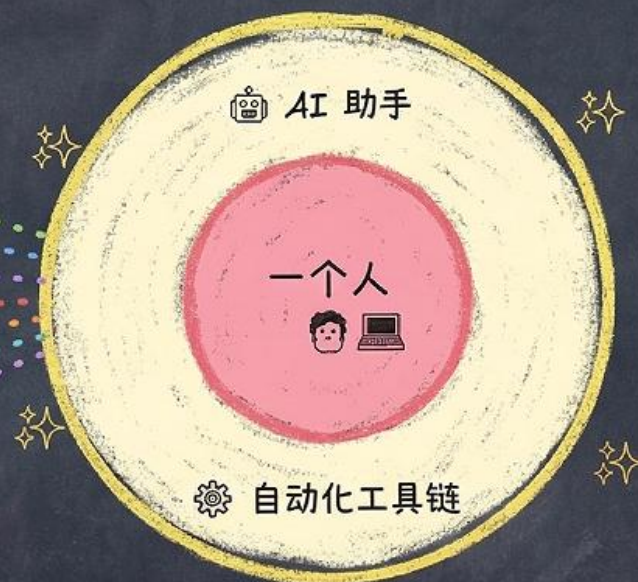
Vibe Coding 可以「造神」

传统 SDLC 7阶段



Vibe Coding

「超级个体」



核心公式

Vibe Coding = 人 + 强大AI助手 + 自动化工具链

从工具能力和实践可能性来看：
7个阶段全部可由 AI 赋能完成

AI 工具能力演进

1) Planning & Requirements

AI 能：

✓ 自动梳理目标

✓ 市场/竞争分析

✓ ROI/PPT/需求文档



产品咨询应用示例：

GPT

Claude

Gemini

2) Design

AI 能：

✓ 架构图 (Mermaid/Excalidraw)

✓ 输出数据模型

✓ API 规范文档



代表工具：

Figma Design to Code

Cursor

→ 输出可调整方案

Planning

Design

Coding

Testing & Deploy

能力演进趋势

AI 工具能力演进

3) Implementation

在实施/执行阶段, AI 主导实现成为事实:

✓ Cursor + ClaudeCode 写模块

✓ AI Test Driven Development

✓ 自动生成 + 自动调试

★ 可验证的超势

代表工具:

Cursor

Claude Code

Codex

Trae

Antigravity



4) Testing / Deployment

CI/CD 工具早已自动化:

GitHub Actions

GitLab CI

自动测试

自动部署

AI 可以:

✓ 编写测试脚本

✓ 分析失败原因

✓ 生成发布脚本



Planning

Design

Implementation

Testing/Deploy

Maintenance

全流程 AI 赋能